Airport Delay Analytics API Technical Report

Student Name: Mingxuan Zhang
Module: COMP3011
Coursework: Coursework 1

For public GitHub repository: AncofordAGGGGG/airport-api: COMP3011_Coursework1
GitHub API documentation:
airport-api/docs/api_documentation.pdf at main · AncofordAGGGGG/airport-api
GitHub PPTX: airport-api/docs/presentation.pptx at main · AncofordAGGGGG/airport-api

1. Introduction

This project implemented a data-based RESTful API for managing flight data and analyzing whether flights are delayed. The system allows users to perform operations such as creating, reading, updating, and deleting flight records, and also enables exploration of connections between airports, and utilizes real flight data to analyze delay situations. Currently, it has the function of viewing the delay rate of a certain airport and the hourly delay rate.
The purpose of this project is to design and develop a well-structured API that can integrate the database and provide data support for airports to improve through data analysis.

2. System Design

The system follows a layered architecture:
- API Layer (FastAPI): handles HTTP requests and responses
- Service Layer (routers): implements business logic
- Database Layer (SQLite + SQLAlchemy): stores and queries flight data

The API is designed following REST principles, with separate endpoints for flights, airports, and analytics.

3. Technology Stack

- Python
Python is main programming language.
- FastAPI
FastAPI was chosen because of its high performance, simplicity, automatic documentation generation and Swagger UI. They are makes testing and documenting the API easier.
- SQLite
SQLite was used as the database.
- SQLAlchemy
SQLAlchemy was used as an Object Relational Mapper to interact with the database.
- Pytest
Pytest was used to implement automated tests for the API.

4. Data Source and Processing

The dataset used in this project is based on the US DOT On-Time Performance dataset (2018–2024). It contains real-world flight records including origin, destination, scheduled departure time, actual departure time, and delay information.
Due to the large file size, the dataset is not included in the repository and must be downloaded separately.

4.1 Data Import

A custom script was developed to import the dataset into the SQLite database. The script performs the following steps:
- Reads CSV data
- Converts time fields into datetime format
- Maps dataset fields to the database schema
- Determines flight status based on delay values

Flights are classified as:
- delayed if delay > 0
- on_time otherwise

5. API Implementation

5.1 CRUD Operations

The API supports full CRUD operations for flight records:

- POST /flights
- GET /flights/{id}
- PUT /flights/{id}
- DELETE /flights/{id}

5.2 Airport Connectivity

The endpoint:
GET /airports/{iata}/destinations
returns all reachable destination airports from a given origin.

5.3 Delay Analytics

Two analytics endpoints were implemented:

5.3.1 Delay Rate

GET /analytics/delay-rate?origin=XXX
This calculates the total number of flights, delayed flights, and the delay rate.

5.3.2 Delay by Hour

GET /analytics/delay-by-hour?origin=XXX
This groups flights by departure hour and calculates delay rates.

6. Authentication

A simple API key-based authentication mechanism was implemented.
To access protected endpoints, the following header must be included:
X-API-Key: 1122332211
If the API key is missing or incorrect, the API returns a 401 Unauthorized response.
This approach was chosen for simplicity while demonstrating basic security concepts.

7. Testing and Validation

The API was tested using FastAPI Swagger UI and automated testing with Pytest.
Automated tests were implemented using FastAPI's TestClient to verify key endpoints, including:
- Flight retrieval and error handling
- Airport destination queries
- Delay analytics endpoints
- Authentication validation

The tests ensure that the API returns correct JSON responses and handles invalid inputs appropriately.

8. Challenges and Solutions

Large Dataset

The dataset size caused performance issues during import. To address this, only a subset of the dataset was used.

Time Conversion

The dataset stores time in HHMM format, which required conversion to standard datetime format. A custom parsing function was implemented to handle this.

Different dataset

At first, I can't found any about the European flight information, the dataset I originally intended to import was about major airports in the UK and the main airports in London, such as Amsterdam Schiphol Airport, Paris Charles de Gaulle Airport, and so on. However, I found it quite difficult for me to locate the corresponding dataset. Eventually, I found one that met my format requirements, which was about the data of major airports in the United States from 2018 to 2024.

9. Limitations and Future Work

Limitations

- The system uses SQLite, which may not scale well for very large datasets
- Only a subset of the dataset is used for performance reasons

Future Work

- The reality provides more information about the flights, such as the check-in counter, boarding gate, luggage counter numbers, etc.
- Complete the import of the real-time flight data API, which will enable users to have a more realistic experience and also increase its practical value. This API has already set up an interface for connecting to the real-time API.
- Deploy the API to a cloud platform
- Improve performance using a more scalable database

10. Conclusion

This project successfully demonstrates the design and implementation of a data-driven RESTful API. It integrates real-world flight data, supports CRUD operations, and provides useful analytics on flight delays, and in the future, preparations will be made to access real-time flight data, thereby achieving commercial value.
This system demonstrates practical operational capabilities in API development, database integration, and data processing.

Appendix

GenAI Usage Declaration

Generative AI tools (ChatGPT， https://chatgpt.com/) were used in this project to assist with debugging, planning and proof-reading.

Having such tools like AI as an aid does indeed enhance efficiency, and it enables me to make the functions I want to achieve a reality within my API, such as the function of synchronizing the display of the world. I am a passionate aviation enthusiast. Therefore, after having access to AI tools, integrating my learning with my hobby has truly excited me. I believe that AI is changing our lives and also transforming computer science.

Reference

Flight Delay Dataset 2018-2024[Online]. [Accessed 11 March 2026].
Available from: Flight Delay Dataset 2018-2024

Conversation Logs

Example:

ChatGPT

README.md
文件

Please proofread these sentences for me

Nice project—your README is already clear and well-structured. I'll proofread it and suggest **grammar, clarity, and consistency improvements** below.

✅ Key Improvements (Quick Summary)

- Fixed small grammar issues (articles, hyphenation, capitalization)
- Improved consistency (e.g., *real-world*, *on-time*, *dataset*)
- Smoothed phrasing for readability
- Standardised formatting in lists and headings

📝 Suggested Edits

1. Title Section

Original:

> A RESTful API built with FastAPI for managing flight data, analysing delay patterns, and exploring airport connectivity using real-world flight datasets.

Improved:

> A RESTful API built with FastAPI for managing flight data, analysing delay patterns, and exploring airport connectivity using real-world flight datasets.

Write the import script:

Bash

```bash
nano scripts/import_flights.py
```

把下面整段代码贴进去：

Python 运行

```python
import csv
from datetime import datetime, timedelta
from pathlib import Path

from app.db import SessionLocal
from app.models import Flight


CSV_FILE = "your_file.csv"   # 改成你的 csv 文件名
IMPORT_LIMIT = 5000          # 先导入 5000 条，稳定一点


def parse_hhmm(value):
    """
    把 hhmm 格式转成 (hour, minute)
    例如:
    930 -> (9, 30)
    5 -> (0, 5)
    2400 / 空值 -> None
    """
    if value is None:
        return None

    value = str(value).strip()
```

Python 运行

```python
                actual_departure=actual_departure,
                status=status
            )

            db.add(flight)
            imported += 1

            if imported % 500 == 0:
                db.commit()
                print(f"Imported {imported} flights...")

        db.commit()

    print(f"Done. Imported: {imported}, Skipped: {skipped}")

except Exception as e:
    db.rollback()
    print("Import failed:", e)

finally:
    db.close()


if __name__ == "__main__":
    main()
```

保存退出：

```
CTRL + O
Enter
CTRL + X
```